BLOOD BANK MANAGEMENT SYSTEM

Software Requirements Specification

UE24CS341A – Software Engineering

Mini-Project Deliverables – Part 1

Nikita Mankani — PES1UG24CS300
Nimay Ballal — PES1UG24CS301
Nishitha S — PES1UG24CS304
Niyatee Singh — PES1UG24CS305

Document Control

Item	Value
Document title	Software Architecture and Design Specification
Document identifier	BBMS SADS 1.0
Version	1.0
Status	Submission copy
Prepared by	Nimay Balal, Nikita Mankani, Niyatee Singh and Nishitha S
Course	UE24CS341A Software Engineering
System scope	Architecture, components, data model, security controls, interactions and APIs

Revision History

Version	Date	Description	Authors
1.0	29 September 2026	Complete part c software architecture and design specification for Part 1 submission	Project team

Approval Record

Role	Name	Signature	Date
Faculty evaluator			
Team representative			

Table of Contents

Document Control
Revision History
Approval Record
Table of Contents
1 Introduction
1.1 Purpose
1.2 Design Scope
1.3 Design Principles
2 Architectural Drivers
3 Architecture Description
3.1 Selected Pattern
3.2 Layer Responsibilities
3.3 Component Diagram
3.4 Deployment View
4 Component Design
4.1 Unit State Machine
4.2 Request State Machine
5 Data Design
5.1 Database Constraints
5.2 Audit Design
6 Security Architecture
6.1 Trust Boundaries
6.2 Authentication and Session Design
6.3 Authorization Design
6.4 Input and Output Protection
6.5 Logging Backup and Recovery
7 Sequence Design
8 API Design
8.1 API Conventions
8.2 Endpoint Summary
8.3 Standard Error Body
9 Error Handling
9.1 Exception Handling Rules
10 Design Traceability
11 Design Decisions and Limitations
References
Appendix A Compliance Checklist

1. Introduction

1.1 Purpose

This specification describes the architecture, components, data model, security controls, interactions, APIs and error-handling design used to satisfy the requirements defined in the Software Requirements Specification, document identifier BBMS SRS 1.0.

1.2 Design Scope

The design covers a Django and PostgreSQL web application deployed behind Nginx. It focuses on a semester-scale system while preserving clear module boundaries and branch-aware data for later growth.

1.3 Design Principles

- Keep business rules outside browser templates and database views.
- Enforce authorization and validation on the server.
- Use transactions for reservation, issue and other multi-record state changes.
- Keep eligibility and shelf-life rules configurable and auditable.
- Expose the minimum information required for each role.

2. Architectural Drivers

Driver	Design response
Sensitive donor and medical data	RBAC, branch scope, HTTPS, minimal disclosure and audit logging
Inventory integrity	State machine, database constraints and transactional reservation and issue
Changing operational rules	Configuration tables and rule service
Semester implementation scope	Modular monolith rather than distributed services
Future branch expansion	Branch identifiers and scoped queries in the initial schema
Testability	Service-layer functions and explicit component interfaces

3. Architecture Description

3.1 Selected Pattern

The system uses a layered modular monolith. Browser-facing views call service-layer modules, service modules apply business rules and data-access code uses the Django object-relational mapper. This pattern is appropriate for a small team because deployment remains simple while domain boundaries remain testable.

3.2 Layer Responsibilities

Layer	Responsibilities
Presentation	Django views, templates, forms, API serializers and role-specific navigation

Layer	Responsibilities
Application services	Use-case orchestration, authorization checks and transaction boundaries
Domain services	Eligibility, unit lifecycle, compatibility, request workflow and alert rules
Data access	Django models, query services, constraints and migrations
Infrastructure	Email, scheduled tasks, logs, backups, Nginx and database connections

3.3 Component Diagram

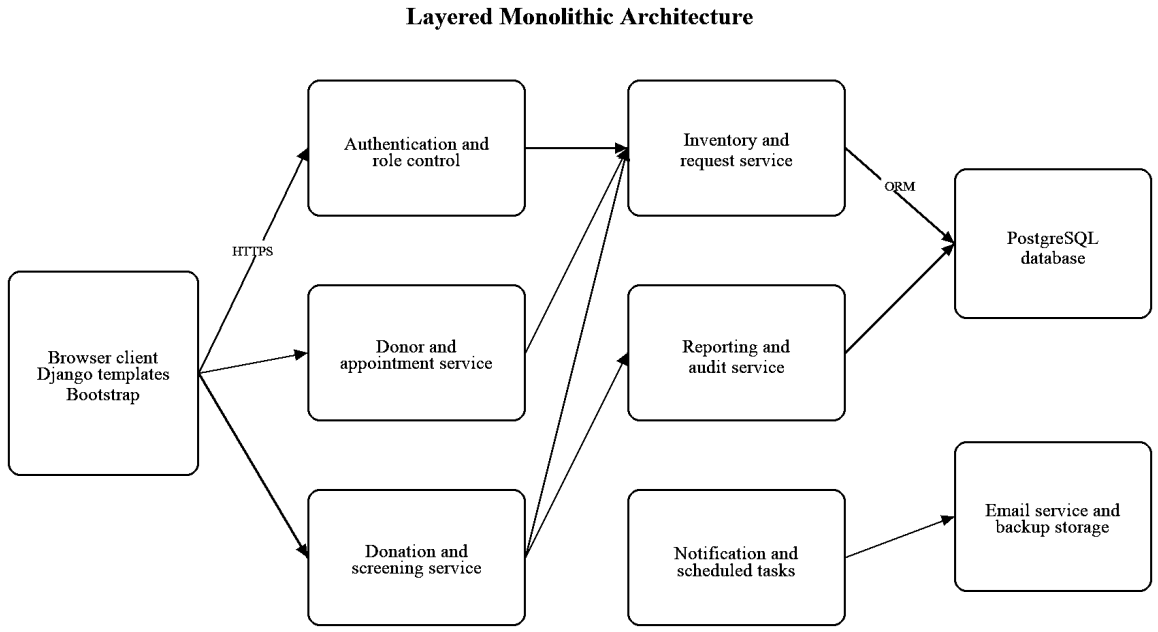


Figure 1 Component diagram

3.4 Deployment View

A browser connects to Nginx over HTTPS. Nginx forwards application traffic to Gunicorn workers running Django. Django uses a restricted database account to connect to PostgreSQL. Scheduled tasks share application code but run under a separate process. Email and backup storage use distinct restricted credentials.

4. Component Design

Component	Main responsibilities	Requirement allocation
Authentication and Role Control	Login, lockout, sessions, account status and authorization	FR-25, FR-26, SEC-01 to SEC-05, SEC-10
Donor and Appointment Service	Profiles, eligibility, deferrals, slots and donor history	FR-01 to FR-08
Donation and Screening Service	Collection, components, test panel and quarantine	FR-09 to FR-13

Component	Main responsibilities	Requirement allocation
Inventory and Request Service	Unit states, expiry, search, requests, reservation, cross-match and issue	FR-14 to FR-24
Administration Service	Branches, configurable rules and thresholds	FR-27
Reporting and Audit Service	Reports and append-only application audit events	FR-28, FR-30, SEC-07
Notification and Task Service	Email, reminders, alerts, expiry task and backup trigger	FR-18, FR-29, NFR-04

4.1 Unit State Machine

Current state	Permitted next states	Condition
Collected	Testing or Discarded	Testing begins or collection is rejected
Testing	Available, Quarantined or Discarded	Mandatory panel and review determine outcome
Available	Reserved, Expired or Discarded	Request allocation, time expiry or authorized disposal
Reserved	Available, Issued or Expired	Reservation release, successful issue or expiry
Issued	None	Terminal state
Quarantined	Discarded	Authorized disposal
Expired	Discarded	Authorized disposal
Discarded	None	Terminal state

4.2 Request State Machine

Current state	Permitted next states
Pending	Approved, Rejected or Cancelled
Approved	Partially Fulfilled, Fulfilled or Cancelled
Partially Fulfilled	Fulfilled or Cancelled
Fulfilled	None
Rejected	None
Cancelled	None

5. Data Design

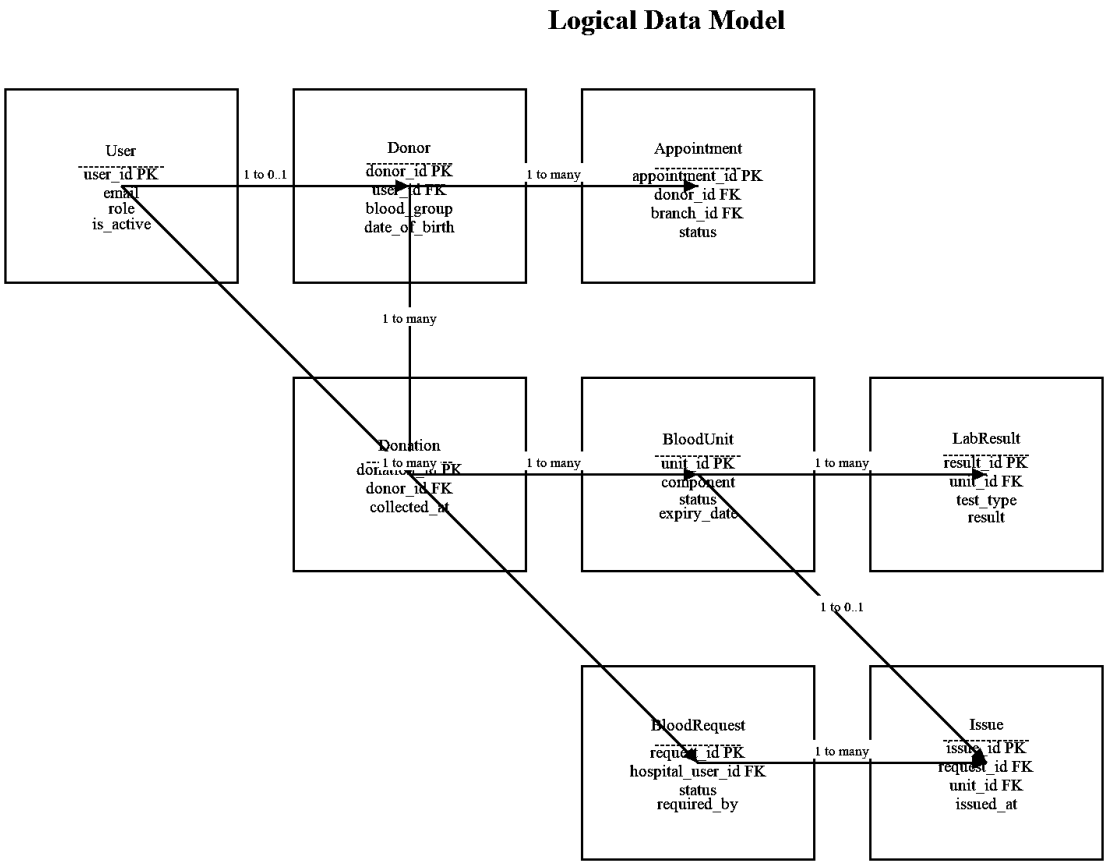


Figure 2 Logical data model

5.1 Database Constraints

- Unique constraints protect account email, donation ID and unit ID.
- Foreign keys prevent orphaned donation, result, request and issue records.
- Database checks restrict enumerated state values and positive quantities.
- A conditional unique constraint prevents more than one active reservation for a unit.
- Transactional service methods lock candidate unit rows during reservation and issue.

5.2 Audit Design

AuditEvent stores actor, branch, event type, affected record type, affected record ID, UTC timestamp, outcome and a structured summary of changed fields. Application roles have no update or delete path for this table. Database and backup access remain restricted to service accounts and designated administrators.

6. Security Architecture

6.1 Trust Boundaries

- The public network ends at Nginx, which terminates HTTPS and forwards only expected traffic.
- The Django application is the only normal path to the database.

- Email and backup services use separate credentials and receive only required data.
- Browser input is untrusted and is validated at form, serializer and service boundaries.

6.2 Authentication and Session Design

Django authentication provides password hashing and session management. The login service applies generic errors and account lockout. Session middleware applies inactivity timeout. Deactivation invalidates active sessions through a recorded account-status change.

6.3 Authorization Design

Each application service checks the account role, active status and branch scope before reading or changing protected records. Templates hide unauthorized actions, but server checks remain authoritative. Object-level filters prevent access by guessed identifiers.

6.4 Input and Output Protection

Django forms and serializers validate type, length, range and permitted values. CSRF tokens protect browser state changes. Templates use automatic output escaping. File upload is outside the initial release, which reduces the attack surface.

6.5 Logging Backup and Recovery

Security events and operational changes create structured audit records without passwords or unnecessary medical details. Backups are encrypted, access-controlled and periodically restored in the acceptance environment. Restoration evidence includes completion time and record comparison.

7. Sequence Design

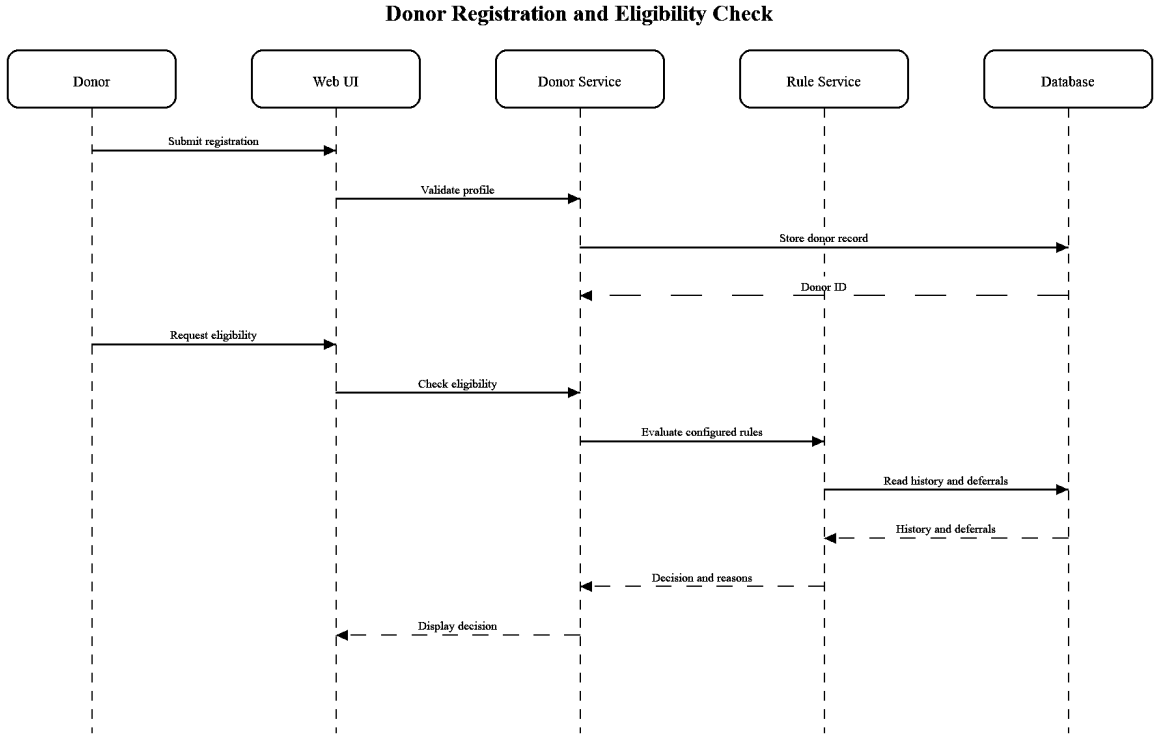


Figure 3 Sequence diagram for registration and eligibility

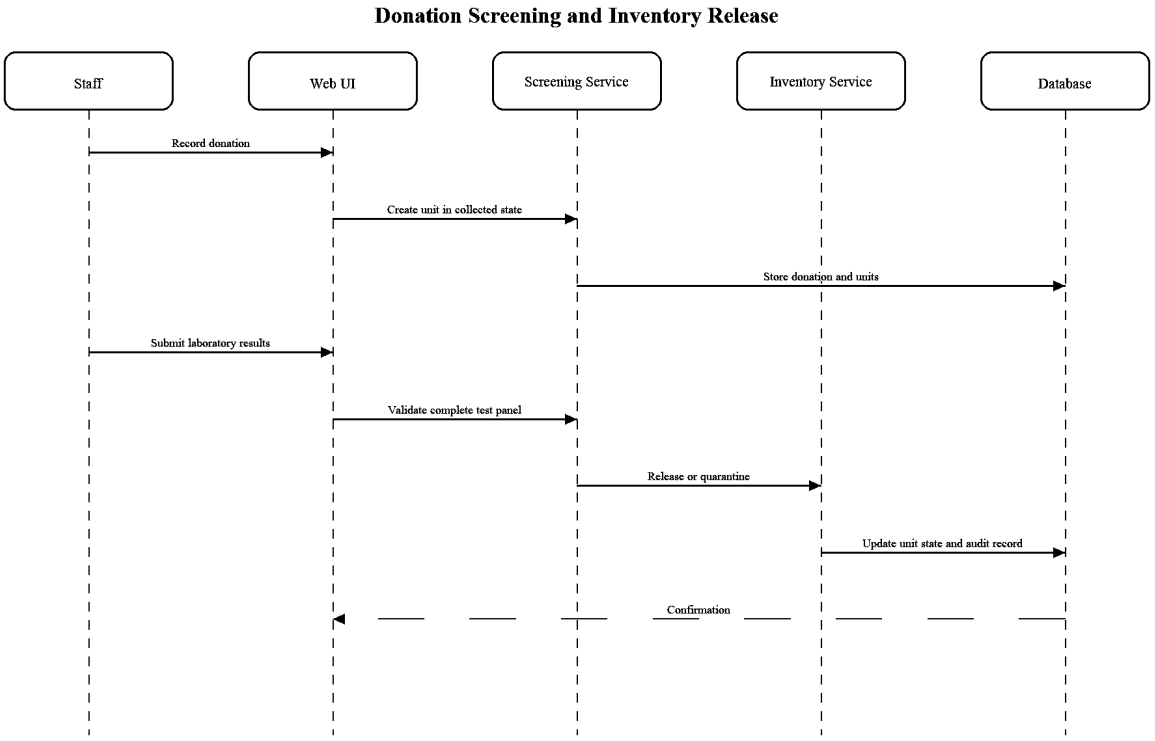


Figure 4 Sequence diagram for screening and inventory release

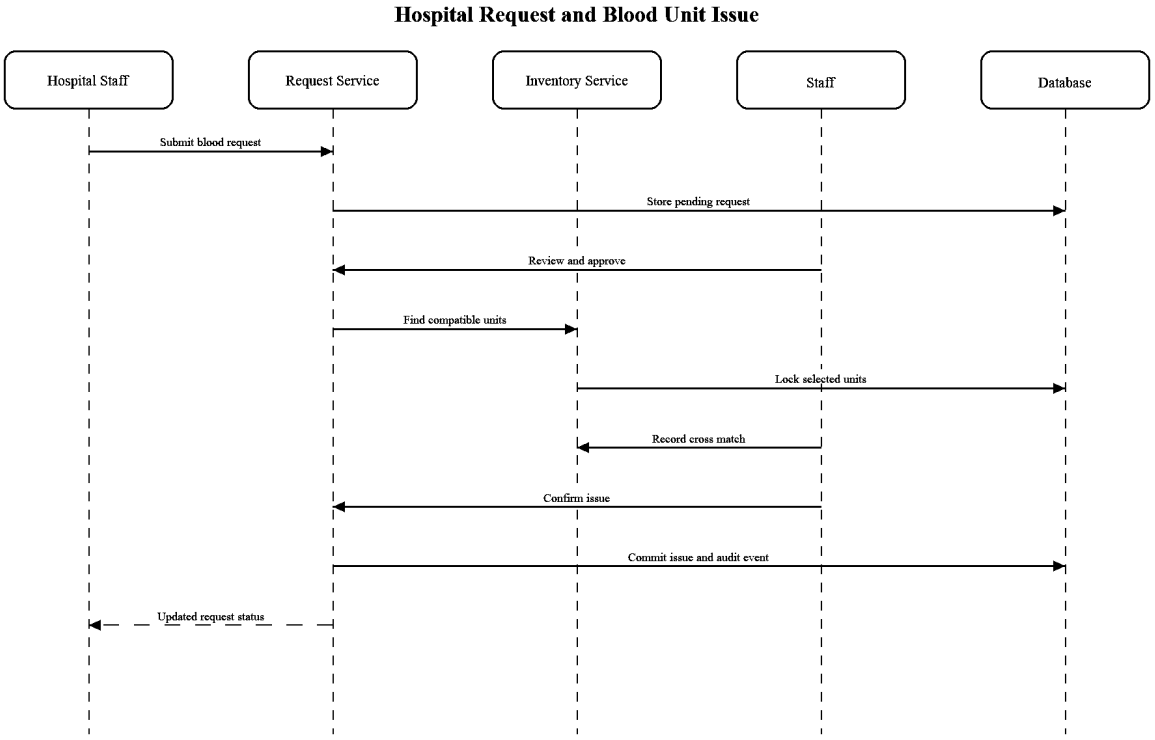


Figure 5 Sequence diagram for request and issue

8. API Design

8.1 API Conventions

- The base path is /api/v1 and payloads use JSON.
- Authentication uses the same secure session as the web application for the first release.
- State-changing requests require CSRF protection and a valid authorized session.
- Validation errors use a stable error code and field-level details.
- Reservation and issue endpoints accept an idempotency key to reduce duplicate processing.

8.2 Endpoint Summary

Table 1 API endpoint summary

Method	Path	Role	Purpose	Responses
POST	/api/v1/auth/login	Public	Authenticate an account and create a session	200, 400, 401, 423
POST	/api/v1/donors	Public	Register a donor profile	201, 400, 409
GET	/api/v1/donors/{id}/eligibility	Donor or Staff	Return eligibility decision and reasons	200, 403, 404
POST	/api/v1/appointments	Donor	Create an appointment	201, 400, 409
PATCH	/api/v1/appointments/{id}	Donor or Staff	Reschedule or cancel an appointment	200, 400, 403, 409
POST	/api/v1/donations	Staff	Record a donation and component units	201, 400, 403, 409
POST	/api/v1/units/{id}/lab-results	Staff	Record a screening result	201, 400, 403, 409
GET	/api/v1/inventory	Staff or Hospital Staff	Search authorized stock information	200, 400, 403
POST	/api/v1/requests	Hospital Staff	Create a blood request	201, 400, 403
PATCH	/api/v1/requests/{id}/status	Staff	Approve, reject or cancel a request	200, 400, 403, 409
POST	/api/v1/requests/{id}/reserve	Staff	Reserve compatible units atomically	200, 400, 403, 409
POST	/api/v1/issues	Staff	Issue cross-matched units	201, 400, 403, 409
GET	/api/v1/reports/inventory	Admin or Staff	Generate an inventory report	200, 400, 403

8.3 Standard Error Body

An API error contains code, message, field_errors when relevant, correlation_id and timestamp. Messages do not expose stack traces, database details, password status or the existence of an unregistered account.

9. Error Handling

Condition	System response	HTTP or UI result	Audit or log
Invalid field input	Reject before service execution	400 with field errors	Application log for repeated abuse only
Unauthenticated access	Do not execute operation	401 or login redirect	Authentication event
Unauthorized access	Do not disclose protected record	403 with generic message	Security audit event
Missing record	Return stable not-found response	404	Normal request log
State or concurrency conflict	Rollback and return current state	409	Operational audit event
Email failure	Record failure and schedule bounded retry	Primary transaction remains committed	Integration log
Database failure	Rollback transaction and show correlation ID	500 with generic message	Error log and administrator alert

9.1 Exception Handling Rules

- Domain exceptions represent validation, authorization, state-transition and conflict errors.
- Views translate known exceptions into stable user messages and status codes.
- Unexpected exceptions are logged once with a correlation ID and produce a generic response.
- Transactions roll back whenever one step of reservation or issue fails.
- No exception message contains a password, secret, full medical history or database query.

10. Design Traceability

Requirement group	Primary components	Design evidence
FR-01 to FR-08	Donor and Appointment Service	Use case UC-01, state validation and Figure 3
FR-09 to FR-13	Donation and Screening Service	UC-02, UC-03 and Figure 4
FR-14 to FR-24	Inventory and Request Service	State tables, UC-04, UC-05 and Figure 5
FR-25 to FR-30	Authentication, Administration, Reporting and Audit	Role matrix, API and error-handling design
NFR-01 to NFR-12	All layers and deployment	Performance limits, constraints, backup and compatibility design
SEC-01 to SEC-10	Security architecture	Sections 6.1 to 6.5 and TC-S01 to TC-S03

11. Design Decisions and Limitations

- A modular monolith avoids the operational cost of distributed services while retaining service boundaries.
- PostgreSQL is selected to remove the earlier unresolved database choice and to support constraints and transactions.
- Clinical analyser integration, barcode printing and national identity integration remain future work.
- The design requires professional approval of eligibility, shelf-life and retention configuration before operational use.

References

[1] ISO IEC IEEE 29148 2018, Systems and software engineering, Life cycle processes, Requirements engineering.

[2] IEEE 829 2008, Standard for Software and System Test Documentation, used as the course-aligned document structure. The standard has been superseded by the ISO IEC IEEE 29119 series.

[3] IEEE 1016 2009, Standard for Information Technology, Systems Design, Software Design Descriptions.

[4] Ministry of Health and Family Welfare, Government of India, National Standards for Blood Centres, Annexure 4 Criteria for Selection of Blood Donor.

[5] Central Drugs Standard Control Organization, Drugs and Cosmetics Act 1940 and Rules 1945, Schedule F provisions for blood banks.

[6] eRaktKosh, Ministry of Health and Family Welfare, overview of centralized donor, testing, inventory and notification functions.

[7] Django Software Foundation, Django documentation for authentication, security and database transactions.

Appendix A Compliance Checklist

Deliverable requirement	Location in this document	Status
Component diagram and description	Section 3.3, Figure 1 and Section 4	Complete
Architecture pattern	Section 3.1	Complete
Architecture traceability	Section 10	Complete
Security architecture	Section 6	Complete
At least two UML sequence diagrams	Section 7, Figures 3 to 5	Complete
API design	Section 8	Complete
Error handling	Section 9	Complete